

Step 1: Mounting

```
from google.colab import drive
drive.mount('/content/drive')
```

[Show hidden output](#)

Step 2: Import Packages

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import string
import re
```

Step 3: Import file

```
read_csv('/content/drive/MyDrive/Colab Notebooks/AnjunaBeach.csv')
```

[Show hidden output](#)

Step 4: Convert it in Regular Expression (Format the Rating)

```
df["Rating"]=df["Rating"].str.extract("bubble_(\d+)").astype(float)
df
```

[Show hidden output](#)

```
df["Date"]=df["Date"].str.replace("Date of experience:", "", regex=False)
df
```

[Show hidden output](#)

```
df["Year"]=df["Date"].str.extract("(20\d{2})")
df
```

[Show hidden output](#)

Step 5: Combine Title and Review

```
df["Text"]= df["Review"].fillna("")+". "+ df["Title"].fillna("")  
print(df["Text"].iloc[0])
```

Beautiful beach. This is one of the beautiful and most sort after be

Step 7: Sentiment analysis

```
!pip install textblob  
from textblob import TextBlob
```

[Show hidden output](#)

```
df["Sentiment_Score"]=df["Text"].apply(lambda x: TextBlob(x).sentim  
df
```

[Show hidden output](#)

```
df.loc[df["Sentiment_Score"]>0,"Sentiment"]="Positive"  
df.loc[df["Sentiment_Score"]<0,"Sentiment"]="Negative"  
df.loc[df["Sentiment_Score"]==0,"Sentiment"]="Neutral"  
df
```

[Show hidden output](#)

Step 8:

```
year_sentiment=df.groupby(["Year","Sentiment"]).size().reset_index(  
year_sentiment
```

[Show hidden output](#)

```
import holoviews as hv  
from holoviews import opts  
hv.extension('bokeh')
```



```

links=year_sentiment[["Year","Sentiment","Count"]]
hv.Chord(links).opts(
    opts.Chord(
        labels="index",
        width=700,
        height=700,
        edge_color="Sentiment",
        edge_cmap=["green","red","yellow"],
        edge_alpha=0.85
    )
)

```

[Show hidden output](#)

```

import matplotlib.pyplot as plt

# Create a stacked bar chart
pivot_df = year_sentiment.pivot(index='Year', columns='Sentiment',
pivot_df[['Positive', 'Neutral', 'Negative']].plot(kind='bar', stac

plt.title('Sentiment Analysis Over Years')
plt.xlabel('Year')
plt.ylabel('Number of Reviews')
plt.xticks(rotation=45)
plt.legend(title='Sentiment')
plt.tight_layout()
plt.show()

```

[Show hidden output](#)

```

# Prepare data for Sankey diagram. The current year_sentiment data
# if we consider Year as a 'source' and Sentiment as a 'target' for
# However, Sankey diagrams typically visualize flow *between* disti
# For sentiment analysis over years, we can visualize the 'flow' of

# Let's create a DataFrame suitable for Sankey, where 'source' coul
# Alternatively, we can show the distribution of sentiment within e

# Option 1: Visualize 'flow' from Year to Sentiment
# This means each year acts as a source node and each sentiment act
# The 'links' data should have 'source', 'target', and 'value' colu

# Create a unique identifier for each year and sentiment combinatio
links_sankey = year_sentiment.copy()
links_sankey['source'] = links_sankey['Year'].astype(str)
links_sankey['target'] = links_sankey['Sentiment']
links_sankey['value'] = links_sankey['Count']

# In Holoviews Sankey, nodes are implicitly created from source/tar
# Let's visualize this.

hv.Sankey(links_sankey, kdims=['source', 'target'], vdims=['value']
          opts.Sankey(height=600, width=900, label_position='right', edge
)

```

[Show hidden output](#)

```

from collections import Counter

all_word = " ".join(df['Text']).split()
Counter(all_word).most_common(10)

```

```

[('the', 1833),
 ('and', 1484),
 ('beach', 1316),
 ('is', 1287),
 ('to', 1183),
 ('of', 995),
 ('a', 913),
 ('in', 693),
 ('for', 542),
 ('you', 448)]

```

```
!pip install gensim
```

```
Collecting gensim
```

```
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylin  
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/pytho  
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/pythor  
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/p  
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/di  
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux  
27.9/27.9 MB 58.2 MB/s e
```

```
Installing collected packages: gensim
```

```
Successfully installed gensim-4.4.0
```

```
import nltk ###Text Preprocessing  
from nltk.corpus import stopwords ###Text Preprocessing stopwords  
from nltk.stem import WordNetLemmatizer ### Text Preprocessing Lemm  
from gensim.models import LdaModel ### LDA Model  
from gensim import corpora ### LDA Corpora  
from gensim.models.coherencemodel import CoherenceModel  
nltk.download('stopwords') ### Downloading Stopwords  
nltk.download('wordnet')
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
```

```
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
```

```
True
```

```
## Lower Case ##
```

```
df['Text'] = df['Text'].str.lower()
```

```
## Remove symbol and Number ##
```

```
df['Text']=df['Text'].str.replace('[^a-z\s]','',regex=True)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\s'
```

```
<>:2: SyntaxWarning: invalid escape sequence '\s'
```

```
/tmp/ipython-input-909067403.py:2: SyntaxWarning: invalid escape sec
```

```
df['Text']=df['Text'].str.replace('[^a-z\s]','',regex=True)
```

✓ REMOVE STOPWORDS

```
domain_stopwords = {  
    "good", "nice", "best", "one", "place", "visit", "time",  
    "like", "really", "much", "many", "great", "beautiful",  
    "goa", "anjuna", "beach"  
}
```

```

stop_words = set(stopwords.words('english')).union(domain_stopwords)
Lemmatizer = WordNetLemmatizer()

def clean_text(text):
    return[
        Lemmatizer.lemmatize(word)
        for word in text.split()
        if word not in stop_words and len(word)>2
    ]
df['tokens'] = df['Text'].apply(clean_text)

```

```
df = df[df['tokens'].str.len()>=4]
```

```

dictionary = corpora.Dictionary(df['tokens'])
dictionary.filter_extremes(no_below=5, no_above=0.3)
corpus = [dictionary.doc2bow(text) for text in df['tokens']]

```

```

num_topics = 3
lda_model = LdaModel(
    corpus=corpus,
    id2word=dictionary,
    num_topics=num_topics,
    random_state=42,
    passes=30,
    alpha="asymmetric",
    eta="auto"
)

```

```

print("\nLDA Topics:\n")
for i, topic in lda_model.print_topics(num_topics=num_topics, num_w
    print(f"Topic {i}: {topic}")

```

LDA Topics:

```

Topic 0: 0.022*"beach" + 0.021*"shack" + 0.021*"crowded" + 0.019*"wa
Topic 1: 0.058*"rocky" + 0.055*"rock" + 0.025*"view" + 0.023*"sunset
Topic 2: 0.079*"market" + 0.040*"flea" + 0.035*"wednesday" + 0.019*"

```

```

coherencemodel = CoherenceModel(model=lda_model, texts=df['tokens']
coherence_score = coherencemodel.get_coherence()
print(f"Coherence Score: {coherence_score}") ## <0.3 = poor, >0.6

```

Coherence Score: 0.3439436591397353

Word Cloud

